

Project: [ServiceName] — Go Microservice

Overview

[Brief description of this service's responsibility]

Tech Stack

- **Runtime:** Go 1.23
- **HTTP Router:** chi v5
- **Database:** PostgreSQL via pgx/v5
- **Testing:** testify v1, golangci-lint
- **Logging:** log/slog (structured)
- **Config:** viper or envconfig

Project Layout

```
cmd/
└─ server/
   └─ main.go           # Entry point – wire dependencies, start server
internal/
├─ domain/             # Types, interfaces, business rules (no deps)
├─ handler/            # HTTP handlers (thin layer, no business logic)
├─ service/            # Business logic / use cases
├─ repository/         # Database implementations
├─ middleware/         # HTTP middleware
pkg/                   # Reusable public packages (used by other services)
migrations/           # SQL migration files (goose format)
go.mod
go.sum
Makefile
Dockerfile
```

Code Standards

Error Handling

- Every error must be handled — never `_`
- Wrap errors with context: `fmt.Errorf("service.GetUser: %w", err)`
- Define sentinel errors for expected cases: `var ErrNotFound = errors.New("not found")`
- Use `errors.Is()` and `errors.As()` for checking

Context

- `context.Context` is ALWAYS the first parameter of any function that does I/O
- Check context cancellation in long-running loops

Concurrency

- Use `sync.WaitGroup` or `errgroup.Group` for concurrent work

- Always close channels in the sender goroutine
- No goroutines that outlive their parent context

Naming

- Short, clear names: `svc` , `repo` , `h` for handler receivers
- Exported types: `UserService` , `OrderRepository`
- Interfaces: describe capability — `Reader` , `Writer` , `UserFinder`
- Tests: `Test[FuncName]_[Scenario]`

HTTP Handlers

- Handlers only: parse input → call service → write response
- No business logic in handlers
- Always return proper HTTP status codes
- Log errors at handler level with request context

Common Commands

```
# Run
go run ./cmd/server

# Test
go test ./...
go test -race ./...

# Lint
golangci-lint run

# Build
go build -o bin/server ./cmd/server

# Migration (goose)
goose -dir migrations postgres "$DATABASE_URL" up

# Generate mocks
go generate ./...
```

Configuration (via environment)

```
DATABASE_URL=postgres://user:pass@localhost:5432/dbname?sslmode=disable
SERVER_ADDR=:8080
LOG_LEVEL=info
```

Do NOT

- Use `init()` functions (except for test helpers)
- Use global mutable variables
- Panic in library code (only in `main`)
- Use `http.DefaultClient` (create clients with timeouts)
- Ignore lint warnings without `//nolint:rulename // reason`